《后端极速转职Golang工程师》

- golang环境安装
  - 源码包下载
    - https://golang.org/dl/ 国外官方
    - https://golang.google.cn/dl/ 中国镜像
    - https://studygolang.com/dl 中文网站
  - linux为例
    - 源码包 go1.14.4.linux-amd64.tar.gz
    - 解压源码包到/usr/local sudo tar -zxvf go1.14.4.linux-amd64.tar.gz -C /usr/local/
    - 配置环境变量
      - 打开~/.bashrc 添加如下配置变量
        - export GOROOT=/usr/local/go
export GOPATH=$HOME/go
export PATH=$PATH:$GOROOT/bin:$GOPATH/bin
        - GOROOT 表示源码包所在路径
        - GOPATH 开发者Go的项目默认路径
      - 不同的操作系统具体配置不同
    - 检测开发环境
      - 手动加载~/.bashrc source ~/.bashrc
      - go version
      - go --help
      - 没有任何错误提示表示环境搭建成功
  - IDE的推荐
    - 收费 Goland
    - 免费 VsCode
    - Vim + go插件
- golang语言特性
  - 优势
    - 简单的部署
      - 可直接编译成机器码可执行
      - 不依赖其他库
      - 直接运行即可部署
    - 静态类型语言
      - 编译的时候即可检查出来隐藏的大多数问题
    - 语言层面的并发
      - 天生基因支持
      - 充分的利用多核
    - 强大的标准库
      - runtime系统调度机制
      - 高效的GC垃圾回收
      - 丰富的标准库
    - 简单易学
      - 25个关键字
      - C语言简洁基因，内嵌C语法支持
      - 面向对象特征
      - 跨平台
    - 大厂领军
  - 适合做什么
    - 云计算基础设施领域
      - docker
      - kubernetes
      - etcd
      - consul
      - cloudflare CDN
      - 七牛云存储
    - 基础后端软件
      - tidb
      - influxdb
      - cockroachdb
    - 微服务
      - go-kit
      - micro
    - 互联网基础设施
      - 以太坊
      - hyperledger
  - 明星作品
    - kubernetes
    - docker
  - 不足
    - 包管理，大部分包都在github上
    - 无泛化类型
    - 所有Excepiton都用Error来处理(比较有争议)。
    - 对C的降级处理，并非无缝，没有C降级到asm那么完美(序列化问题)
- golang语法新奇
  - 从一个main函数初见golang语法

```
package main

import "fmt"

func main() {
        /* 简单的程序 万能的hello world */
        fmt.Println("Hello Go")
}
```

    - golang中的表达式，加";", 和不加 都可以，建议是不加
    - 函数的{ 一定是 和函数名在同一行的，否则编译错
    - package main //程序的包名

```
/*
import "fmt"
import "time"
*/
import (
    "fmt"
    "time"
)
```

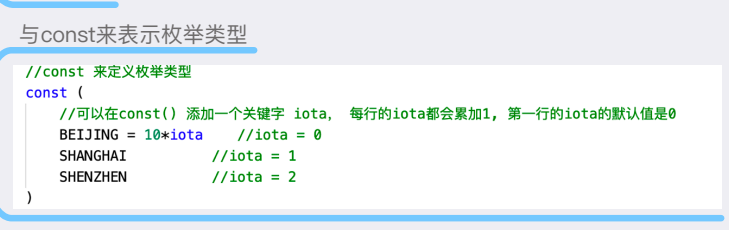
  - 变量
    - 变量的声明
      - 局部变量的声明
        - 方法一 //方法一：声明一个变量 默认的值是0
var a int
        - 方法二 //方法二：声明一个变量，初始化一个值
var b int = 100
        - 方法三 //方法三：在初始化的时候，可以省去数据类型，通过值自动匹配当前的变量的数据类型
var c = 100
        - 方法四(常用) //方法四：(常用的方法) 省去var关键字，直接自动匹配
e := 100
      - 全局变量的声明 方法四不支持全局
      - 多变量的声明
        - 单行写法
          - var xx, yy int = 100, 200
          - var kk, ll = 100, "Aceld"
        - 多行写法

```
var (
        vv int  = 100
        jj bool = true
    )
```

  - 常量与iota
    - 常量
      - const a int = 10

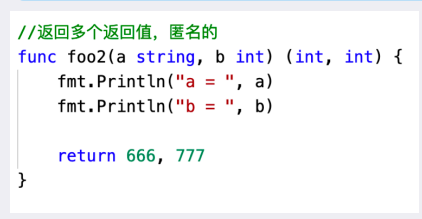
```
const (
   a = 10
   b = 20
)
```

    - iota
      - 与const来表示枚举类型

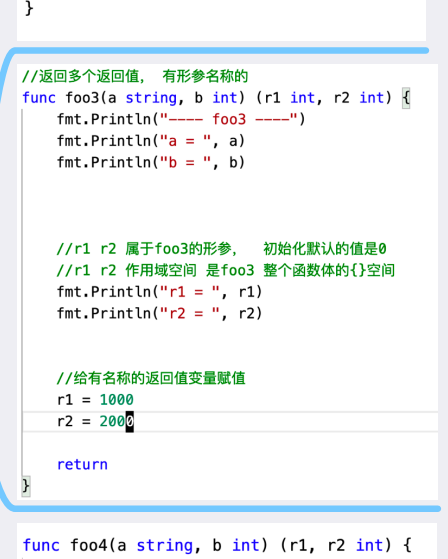


```
//const 来定义枚举类型
const (
    //可以在const() 添加一个关键字 iota， 每行的iota都会累加1，第一行的iota的默认值是0
    BEIJING = 10*iota     //iota = 0
    SHANGHAI          //iota = 1
    SHENZHEN          //iota = 2
)
```

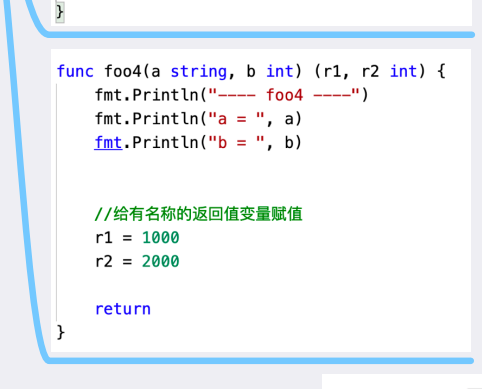
  - 函数
    - 多返回值



```
//返回多个返回值，匿名的
func foo2(a string, b int) (int, int) {
    fmt.Println("a = ", a)
    fmt.Println("b = ", b)

    return 666, 777
}
```

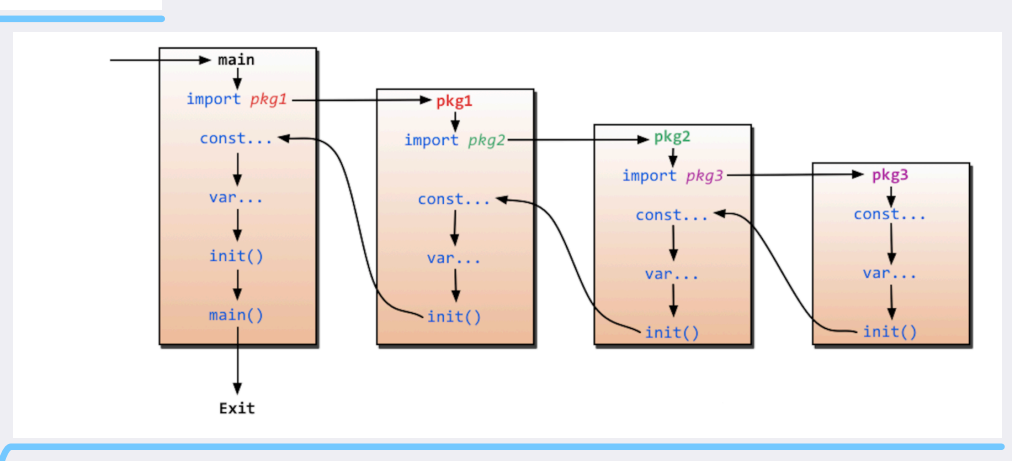


```
//返回多个返回值， 有形参名称的
func foo3(a string, b int) (r1 int, r2 int) {
    fmt.Println("---- foo3 ----")
    fmt.Println("a = ", a)
    fmt.Println("b = ", b)



    //r1 r2 属于foo3的形参，  初始化默认的值是0
    //r1 r2 作用域空间 是foo3 整个函数体的{}空间
    fmt.Println("r1 = ", r1)
    fmt.Println("r2 = ", r2)


    //给有名称的返回值变量赋值
    r1 = 1000
    r2 = 2000

    return
}
```



```
func foo4(a string, b int) (r1, r2 int) {
    fmt.Println("---- foo4 ----")
    fmt.Println("a = ", a)
    fmt.Println("b = ", b)


    //给有名称的返回值变量赋值
    r1 = 1000
    r2 = 2000

    return
}
```

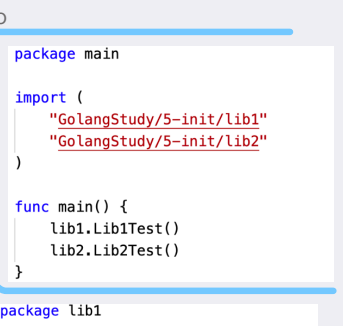
    - init函数与import导包
      - init函数



        - 制作包的时候，项目路径如下

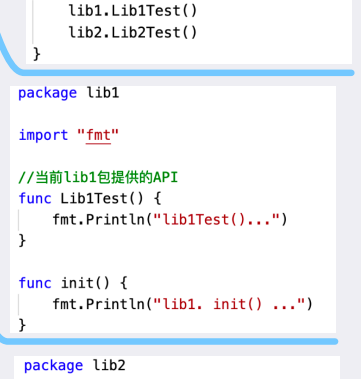
```
$GOPATH/GolangStudy/5-init/
├── lib1/
│   └── lib1.go
├── lib2/
│   └── lib2.go
└── main.go
```

          - main.go



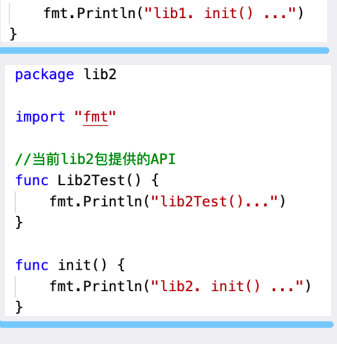
```
package main

import (
    "GolangStudy/5-init/lib1"
    "GolangStudy/5-init/lib2"
)

func main() {
    lib1.Lib1Test()
    lib2.Lib2Test()
}
```

          - lib1.go



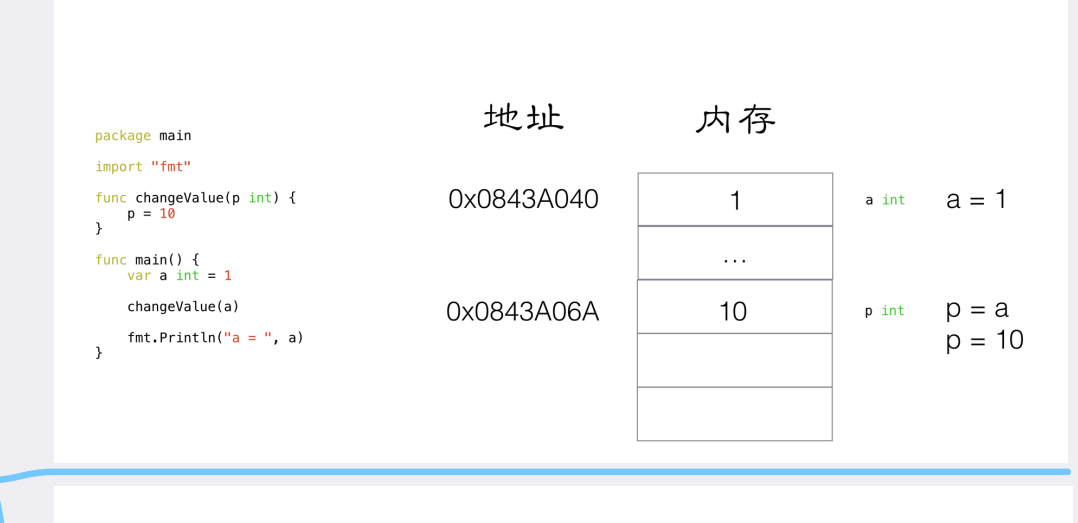
```
package lib1

import "fmt"

//当前lib1包提供的API
func Lib1Test() {
    fmt.Println("lib1Test()...")
}

func init() {
    fmt.Println("lib1. init() ...")
}
```

          - lib2.go

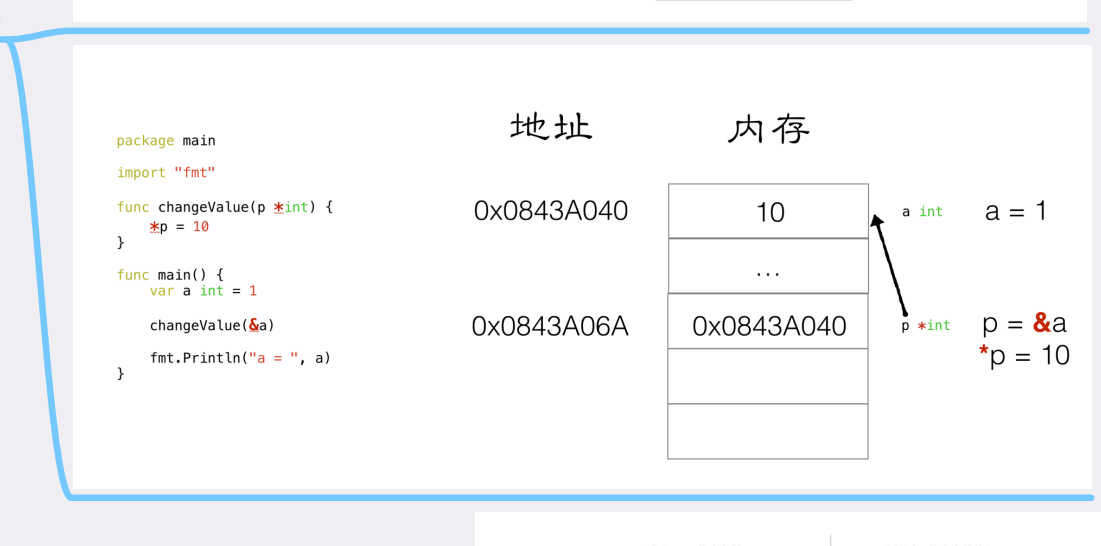


```
package lib2

import "fmt"

//当前lib2包提供的API
func Lib2Test() {
    fmt.Println("lib2Test()...")
}

func init() {
    fmt.Println("lib2. init() ...")
}
```

      - import导包
        - import _ “fmt” 给fmt包起一个别名，匿名， 无法使用当前包的方法，但是会执行当前的包内部的init()方法
        - import aa “fmt” 给fmt包起一个别名，aa， aa.Println()来直接调用。
        - import . “fmt” 将当前fmt包中的全部方法，导入到当前本包的作用中，fmt包中的全部的方法可以直接使用API来调用，不需要fmt.API来调用
  - 指针

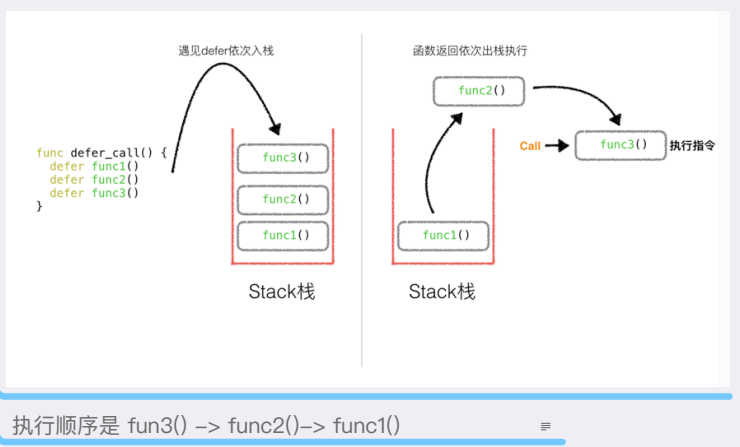


```
package main

import "fmt"

func changeValue(p int) {
    p = 10
}

func main() {
    var a int = 1

    changeValue(a)

    fmt.Println("a = ", a)
}
```



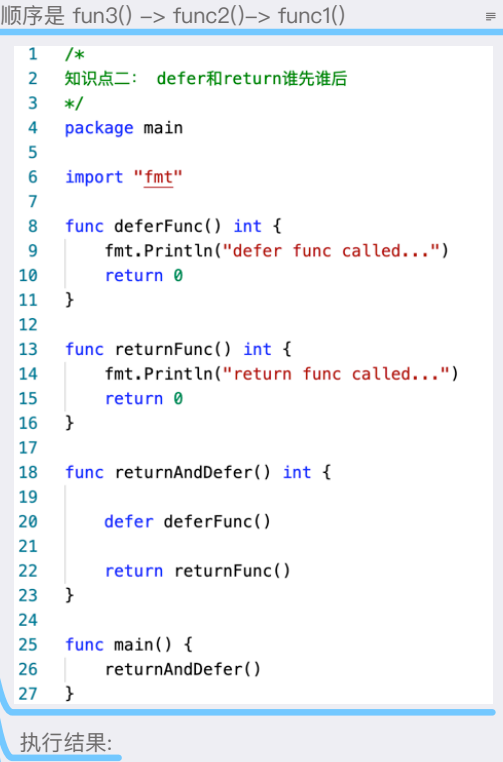
```
package main

import "fmt"

func changeValue(p *int) {
    *p = 10
}

func main() {
    var a int = 1

    changeValue(&a)

    fmt.Println("a = ", a)
}
```

  - defer
    - 知识点1 defer的执行顺序



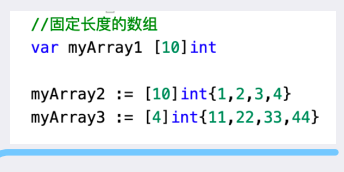
```
func defer_call() {
    defer func1()
    defer func2()
    defer func3()
}
```

      - 执行顺序是 fun3() --> func2()--> func1()
    - 知识点2 defer和return谁先谁后



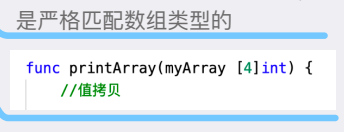
```
/*
知识点二:  defer和return谁先谁后
*/
package main

import "fmt"

func deferFunc() int {
    fmt.Println("defer func called...")
    return 0
}

func returnFunc() int {
    fmt.Println("return func called...")
    return 0
}

func returnAndDefer() int {

    defer deferFunc()

    return returnFunc()
}

func main() {
    returnAndDefer()
}
```

      - 执行结果:
      - 结论: return之后的语句先执行，defer后的语句后执行
  - 切片slice
    - 数组
      - 声明数组的方式



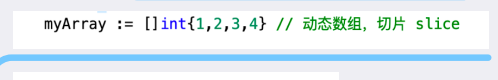
```
//固定长度的数组
var myArray1 [10]int

myArray2 := [10]int{1,2,3,4}
myArray3 := [4]int{11,22,33,44}
```

        - 数组的长度是固定的
        - 固定长度的数组在传参的时候，是严格匹配数组类型的

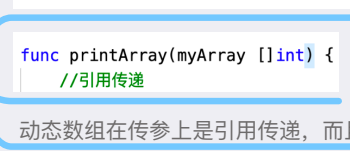


```
func printArray(myArray [4]int) {
    //值拷贝
```

    - slice(动态数组)

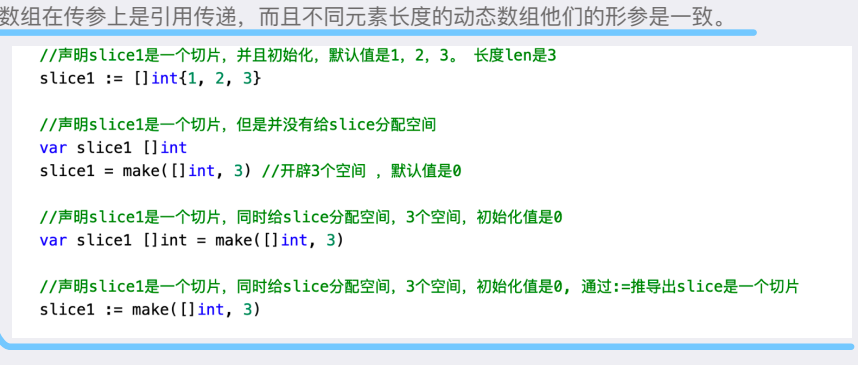


```
myArray := []int{1,2,3,4} // 动态数组，切片 slice
```



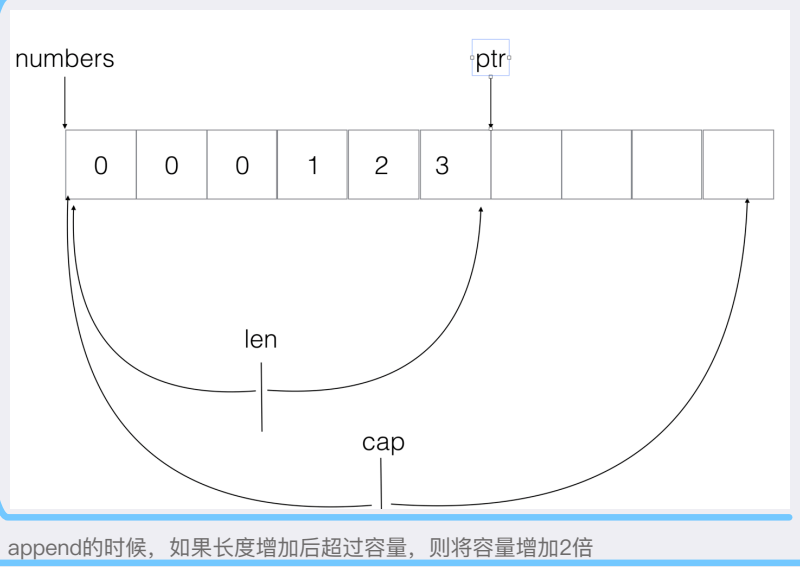
```
func printArray(myArray []int) {
    //引用传递
```

      - 动态数组在传参上是引用传递，而且不同元素长度的动态数组他们的形参是一致。
    - slice
      - 声明方式

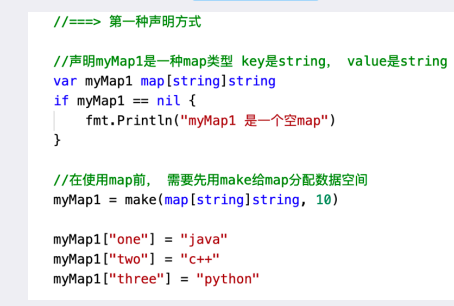


```
//声明slice1是一个切片，并且初始化，默认值是1，2，3。 长度len是3
slice1 := []int{1, 2, 3}

//声明slice1是一个切片，但是并没有给slice分配空间
var slice1 []int
slice1 = make([]int, 3) //开辟3个空间 ，默认值是0

//声明slice1是一个切片，同时给slice分配空间，3个空间，初始化值是0
var slice1 []int = make([]int, 3)

//声明slice1是一个切片，同时给slice分配空间，3个空间，初始化值是0，通过:=推导出slice是一个切片
slice1 := make([]int, 3)
```

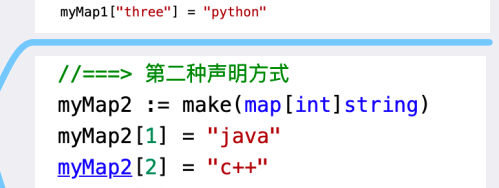
      - 使用方式
        - 切片容量的追加
          - len 和 cap
            - 切片的长度和容量不同，长度表示左指针至右指针之间的距离，容量表示左指针至底层数组末尾的距离。



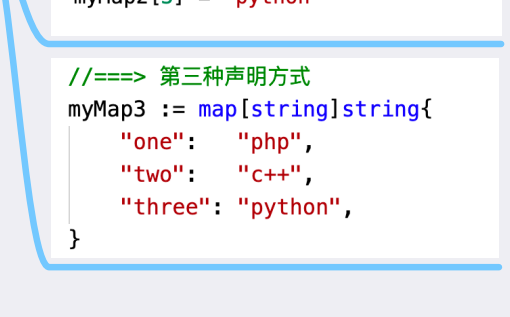
          - 切片的扩容机制，append的时候，如果长度增加后超过容量，则将容量增加2倍
        - 切片的截取
  - map
    - 声明方式



```
//===> 第一种声明方式

//声明myMap1是一种map类型 key是string， value是string
var myMap1 map[string]string
if myMap1 == nil {
    fmt.Println("myMap1 是一个空map")
}

//在使用map前， 需要先用make给map分配数据空间
myMap1 = make(map[string]string, 10)

myMap1["one"] = "java"
myMap1["two"] = "c++"
myMap1["three"] = "python"
```

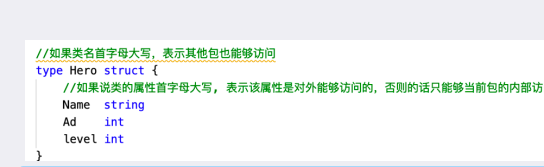


```
//===> 第二种声明方式
myMap2 := make(map[int]string)
myMap2[1] = "java"
myMap2[2] = "c++"
myMap2[3] = "python"
```

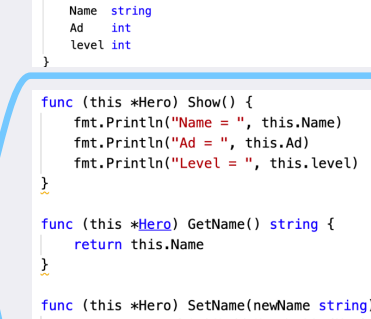


```
//===> 第三种声明方式
myMap3 := map[string]string{
    "one":   "php",
    "two":   "c++",
    "three": "python",
}
```

    - 使用方式
  - 面向对象特征
    - 封装

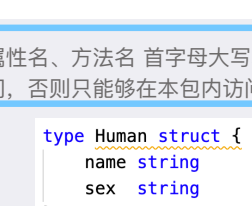


```
//如果类名首字母大写，表示其他包也能够访问
type Hero struct {
    //如果说类的属性首字母大写，表示该属性是对外能够访问的，否则的话只能够当前包的内部访问
    Name  string
    Ad    int
    level int
}
```

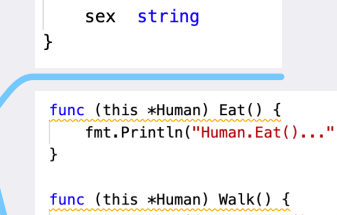


```
func (this *Hero) Show() {
    fmt.Println("Name = ", this.Name)
    fmt.Println("Ad = ", this.Ad)
    fmt.Println("Level = ", this.level)
}

func (this *Hero) GetName() string {
    return this.Name
}

func (this *Hero) SetName(newName string) {
    //this 是调用该方法的对象的一个副本 (拷贝)
    this.Name = newName
}
```

      - 类名、属性名、方法名 首字母大写表示对外(其他包)可以访问，否则只能够在本包内访问
    - 继承
      - 父类

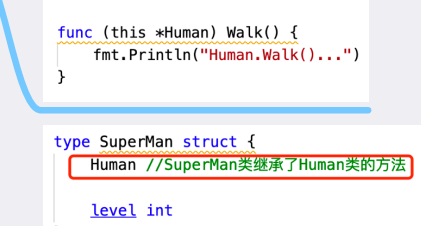


```
type Human struct {
    name string
    sex  string
}
```

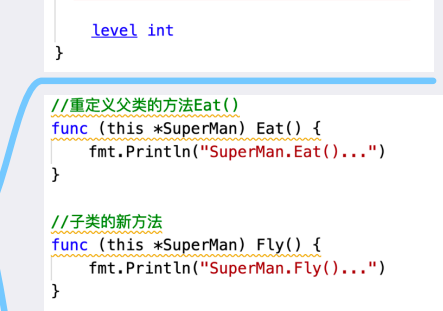


```
func (this *Human) Eat() {
    fmt.Println("Human.Eat()...")
}

func (this *Human) Walk() {
    fmt.Println("Human.Walk()...")
}
```

      - 子类

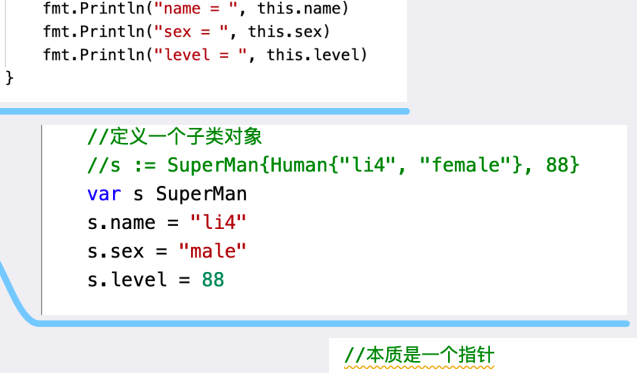


```
type SuperMan struct {
    Human //SuperMan类继承了Human类的方法

    level int
}
```



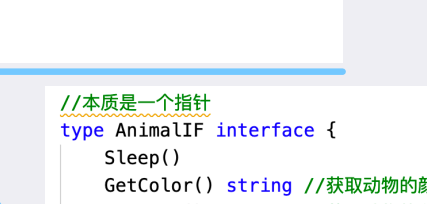
```
//重定义父类的方法Eat()
func (this *SuperMan) Eat() {
    fmt.Println("SuperMan.Eat()...")
}

//子类的新方法
func (this *SuperMan) Fly() {
    fmt.Println("SuperMan.Fly()...")
}

func (this *SuperMan) Print() {
    fmt.Println("name = ", this.name)
    fmt.Println("sex = ", this.sex)
    fmt.Println("level = ", this.level)
}
```

      - 定义子类



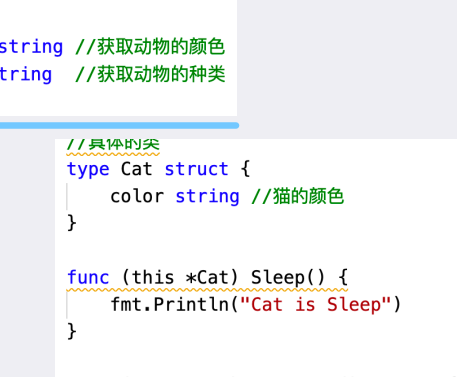
```
//定义一个子类对象
//s := SuperMan{Human{"li4", "female"}, 88}
var s SuperMan
s.name = "li4"
s.sex = "male"
s.level = 88
```

    - 多态
      - 基本的要素
        - 有一个父类(有接口)



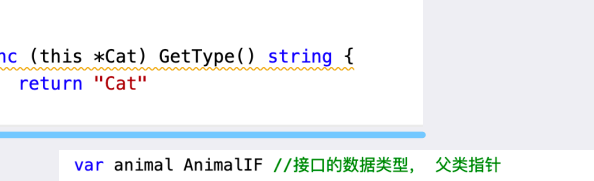
```
//本质是一个指针
type AnimalIF interface {
    Sleep()
    GetColor() string //获取动物的颜色
    GetType() string  //获取动物的种类
}
```

        - 有子类(实现了父类的全部接口方法)



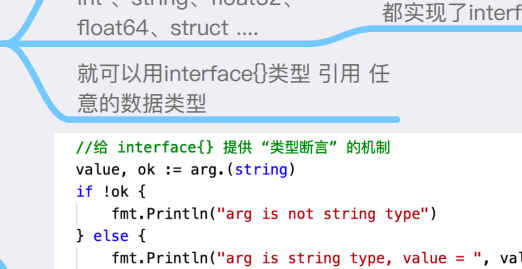
```
//具体的类
type Cat struct {
    color string //猫的颜色
}

func (this *Cat) Sleep() {
    fmt.Println("Cat is Sleep")
}

func (this *Cat) GetColor() string {
    return this.color
}

func (this *Cat) GetType() string {
    return "Cat"
}
```

        - 父类类型的变量(指针) 指向(引用) 子类的具体数据变量



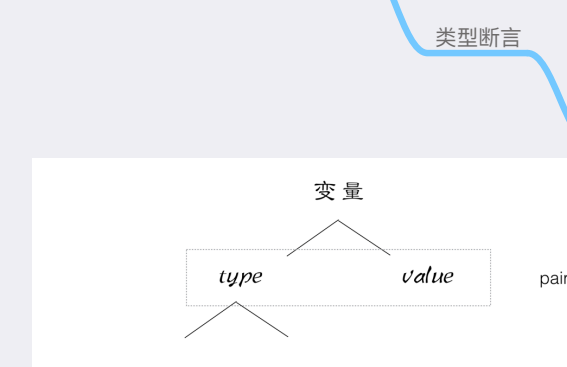
```
var animal AnimalIF //接口的数据类型， 父类指针
animal = &Cat{"Green"}
animal.Sleep() //调用的就是Cat的Sleep()方法 , 多态的现象
```

      - interface
        - 通用万能类型
          - interface{}
            - 空接口
            - int 、string、float32、float64、struct .... 都实现了interface{}
            - 就可以用interface{}类型 引用 任意的数据类型
          - 类型断言

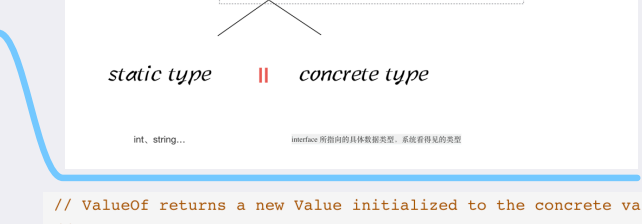


```
//给 interface{} 提供 "类型断言" 的机制
value, ok := arg.(string)
if !ok {
    fmt.Println("arg is not string type")
} else {
    fmt.Println("arg is string type, value = ", value)

    fmt.Printf("value type is %T\n", value)
}
```

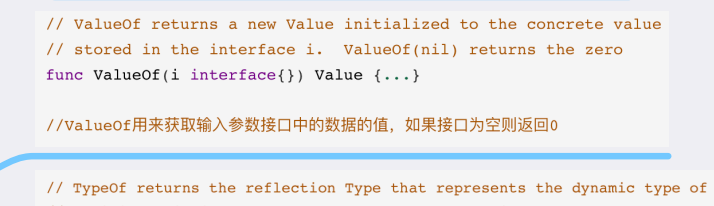
  - 反射
    - 变量的结构



    - reflect包

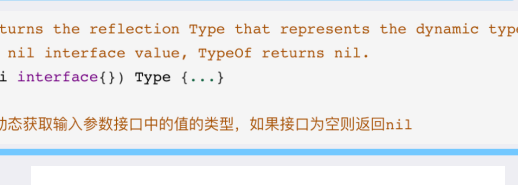


```
// ValueOf returns a new Value initialized to the concrete value
// stored in the interface i.  ValueOf(nil) returns the zero
func ValueOf(i interface{}) Value {...}

//ValueOf用来获取输入参数接口中的数据的值，如果接口为空则返回0
```

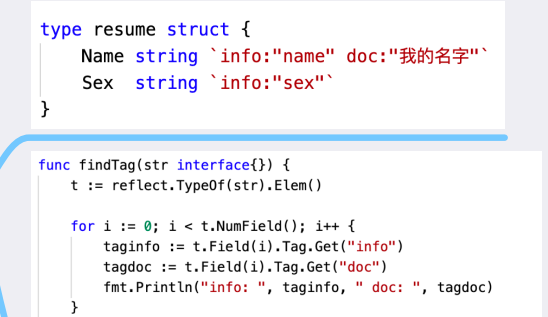


```
// TypeOf returns the reflection Type that represents the dynamic type of i.
// If i is a nil interface value, TypeOf returns nil.
func TypeOf(i interface{}) Type {...}

//TypeOf用来动态获取输入参数接口中的值的类型，如果接口为空则返回nil
```

  - 结构体标签
    - 结构体标签的定义

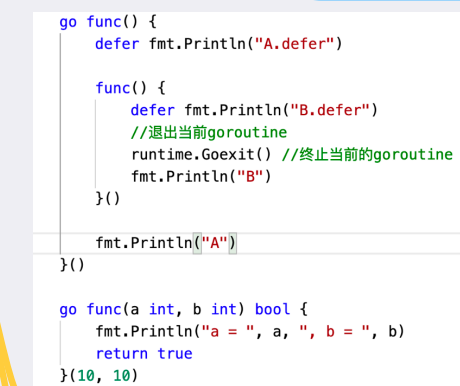


```
type resume struct {
    Name string `info:"name" doc:"我的名字"`
    Sex  string `info:"sex"`
}
```



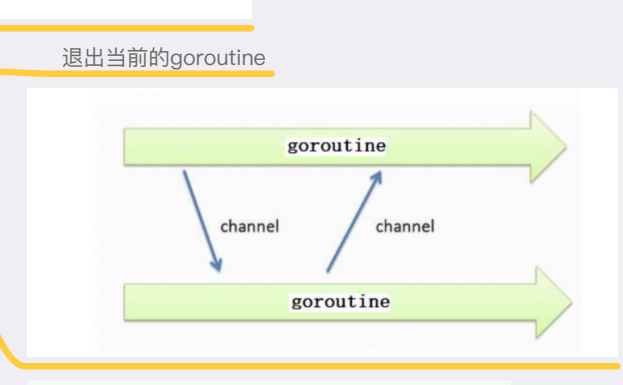
```
func findTag(str interface{}) {
    t := reflect.TypeOf(str).Elem()

    for i := 0; i < t.NumField(); i++ {
        taginfo := t.Field(i).Tag.Get("info")
        tagdoc := t.Field(i).Tag.Get("doc")
        fmt.Println("info: ", taginfo, " doc: ", tagdoc)
    }
}
```

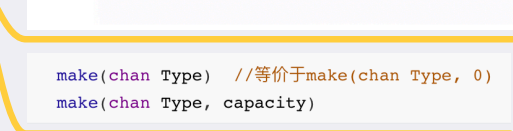
    - 结构体标签应用
      - json编解码
      - orm映射关系
- golang高阶
  - goroutine



```
go func() {
    defer fmt.Println("A.defer")

    func() {
        defer fmt.Println("B.defer")
        //退出当前goroutine
        runtime.Goexit() //终止当前的goroutine
        fmt.Println("B")
    }()

    fmt.Println("A")
}()

go func(a int, b int) bool {
    fmt.Println("a = ", a, ", b = ", b)
    return true
}(10, 10)
```

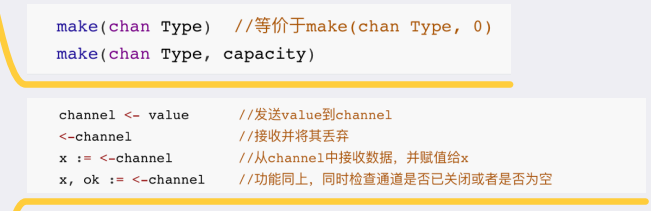
    - runtime.Goexit() 退出当前的goroutine
  - channel
    - channel的定义



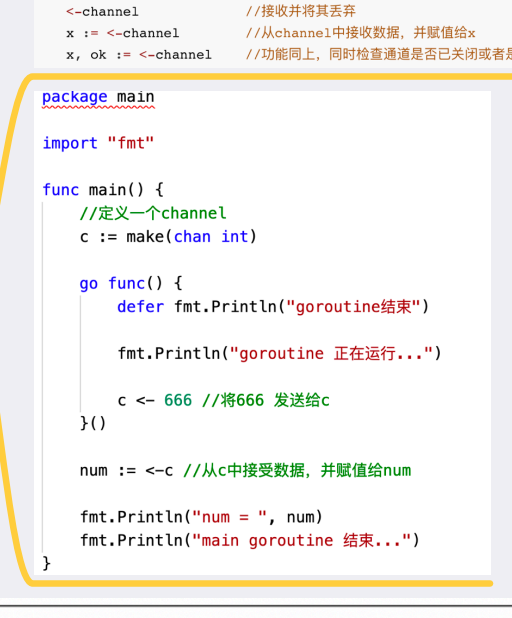


```
make(chan Type)  //等价于make(chan Type, 0)
make(chan Type, capacity)
```

    - channel的使用

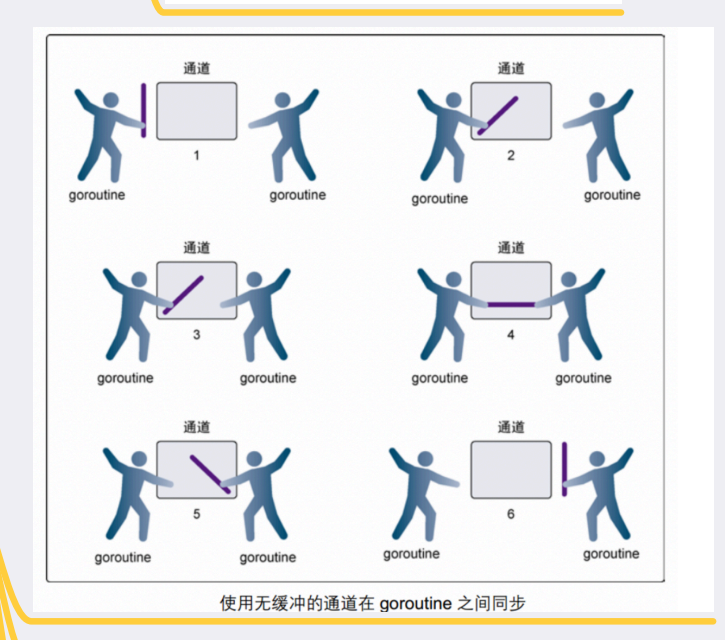


```
channel <- value      //发送value到channel
<-channel             //接收并将其丢弃
x := <-channel        //从channel中接收数据，并赋值给x
x, ok := <-channel    //功能同上，同时检查通道是否已关闭或者是否为空
```



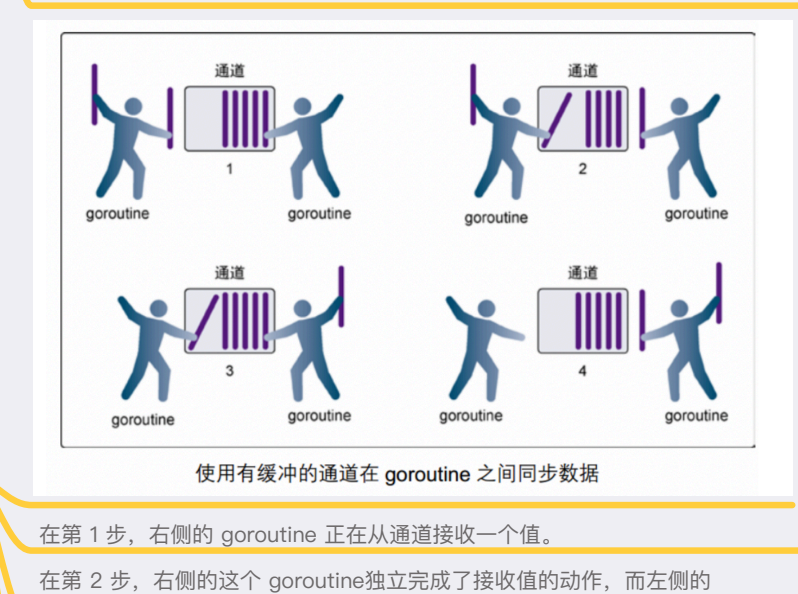
```
package main

import "fmt"

func main() {
    //定义一个channel
    c := make(chan int)

    go func() {
        defer fmt.Println("goroutine结束")

        fmt.Println("goroutine 正在运行...")

        c <- 666 //将666 发送给c
    }()

    num := <-c //从c中接受数据，并赋值给num

    fmt.Println("num = ", num)
    fmt.Println("main goroutine 结束...")
}
```

  - 无缓冲的channel



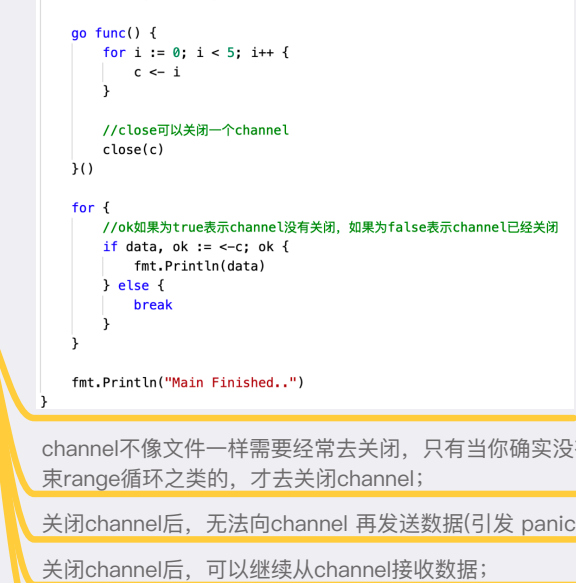
使用无缓冲的通道在 goroutine 之间同步

    - 在第 1 步，两个 goroutine 都到达通道，但哪个都没有开始执行发送或者接收。
    - 在第 2 步，左侧的 goroutine 将它的手伸进了通道，这模拟了向通道发送数据的行为。这时，这个 goroutine 会在通道中被锁住，直到交换完成。
    - 在第 3 步，右侧的 goroutine 将它的手放入通道，这模拟了从通道里接收数据。这个 goroutine 一样也会在通道中被锁住，直到交换完成。
    - 在第 4 步和第 5 步，进行交换，并最终，在第 6 步，两个 goroutine 都将它们的手从通道里拿出来，这模拟了被锁住的 goroutine 得到释放。两个 goroutine 现在都可以去做其他事情了。
  - 有缓冲的channel



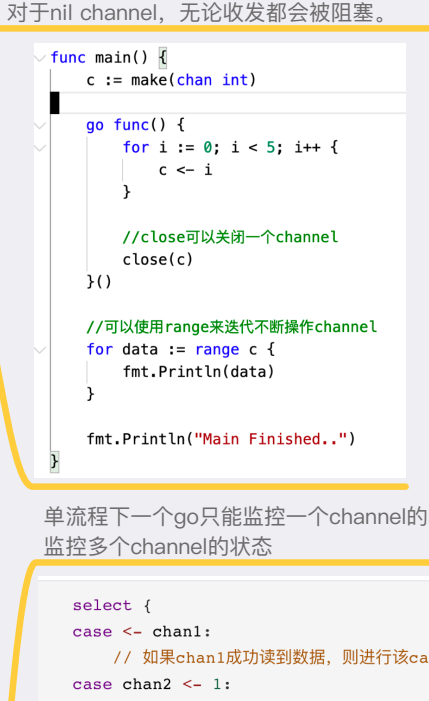
使用有缓冲的通道在 goroutine 之间同步数据

    - 在第 1 步，右侧的 goroutine 正在从通道接收一个值。
    - 在第 2 步，右侧的这个 goroutine独立完成了接收值的动作，而左侧的 goroutine 正在发送一个新值到通道里。
    - 在第 3 步，左侧的goroutine 还在向通道发送新值，而右侧的 goroutine 正在从通道接收另外一个值。这个步骤里的两个操作既不是同步的，也不会互相阻塞。
    - 最后，在第 4 步，所有的发送和接收都完成，而通道里还有几个值，也有一些空间可以存更多的值。
    - 特点
      - 当channel已经满，再向里面写数据，就会阻塞
      - 当channel为空，从里面取数据也会阻塞
  - 关闭channel



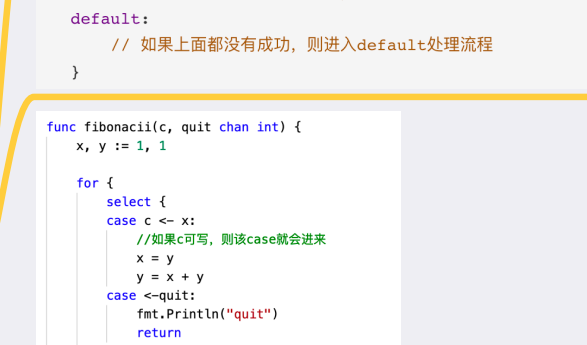
```
func main() {
    c := make(chan int)

    go func() {
        for i := 0; i < 5; i++ {
            c <- i
        }

        //close可以关闭一个channel
        close(c)
    }()

    for {
        //ok如果为true表示channel没有关闭，如果为false表示channel已经关闭
        if data, ok := <-c; ok {
            fmt.Println(data)
        } else {
            break
        }
    }

    fmt.Println("Main Finished..")
}
```

    - channel不像文件一样需要经常去关闭，只有当你确实没有任何发送数据了，或者你想显式的结束range循环之类的，才去关闭channel；
    - 关闭channel后，无法向channel 再发送数据(引发 panic 错误后导致接收立即返回零值)；
    - 关闭channel后，可以继续从channel接收数据；
    - 对于nil channel，无论收发都会被阻塞。
  - channel与range

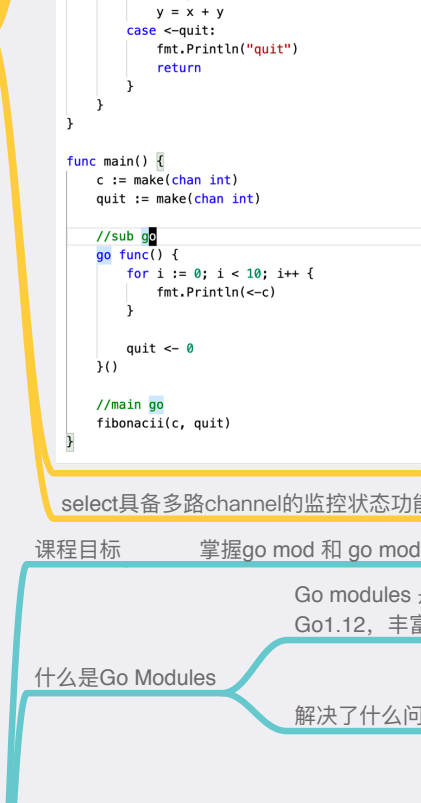


```
func main() {
    c := make(chan int)

    go func() {
        for i := 0; i < 5; i++ {
            c <- i
        }

        //close可以关闭一个channel
        close(c)
    }()

    //可以使用range来迭代不断操作channel
    for data := range c {
        fmt.Println(data)
    }

    fmt.Println("Main Finished..")
}
```

  - channel与select
    - 单流程下一个go只能监控一个channel的状态，select可以完成监控多个channel的状态



```
select {
case <- chan1:
    // 如果chan1成功读到数据，则进行该case处理语句
case chan2 <- 1:
    // 如果成功向chan2写入数据，则进行该case处理语句
default:
    // 如果上面都没有成功，则进入default处理流程
}
```



```
func fibonacii(c, quit chan int) {
    x, y := 1, 1

    for {
        select {
        case c <- x:
            //如果c可写，则该case就会进来
            x = y
            y = x + y
        case <-quit:
            fmt.Println("quit")
            return
        }
    }
}

func main() {
    c := make(chan int)
    quit := make(chan int)

    //sub go
    go func() {
        for i := 0; i < 10; i++ {
            fmt.Println(<-c)
        }

        quit <- 0
    }()

    //main go
    fibonacii(c, quit)
}
```

    - select具备多路channel的监控状态功能
- go modules模块管理
  - 课程目标 掌握go mod 和 go modules 进行项目依赖管理
  - 什么是Go Modules
    - Go modules 是 Go 语言的依赖解决方案，发布于 Go1.11，成长于 Go1.12，丰富于 Go1.13，正式于 Go1.14 推荐在生产上使用。
    - 解决了什么问题
      - Go 语言长久以来的依赖管理问题。
      - “淘汰”现有的 GOPATH 的使用模式。
      - 统一社区中的其它的依赖管理工具（提供迁移功能）。
  - GOPATH的工作模式
    - GOPATH的弊端
      - 无版本控制概念
      - 无法同步一致第三方版本号
      - 无法指定当前项目引用的第三方版本号
  - Go Modules模式
    - go mod命令
      - go mod init 生成 go.mod 文件
      - go mod download 下载 go.mod 文件中指明的所有依赖
      - go mod tidy 整理现有的依赖
      - go mod graph 查看现有的依赖结构
      - go mod edit 编辑 go.mod 文件
      - go mod vendor 导出项目所有的依赖到vendor目录
      - go mod verify 校验一个模块是否被篡改过
      - go mod why 查看为什么需要依赖某模块
    - go mod 环境变量
      - GO111MODULE
        - 是否开启go modules模式
        - 建议go V1.11之后，都设置为on
      - GOPROXY
        - 项目的第三方依赖库的下载源地址
        - 建议设置国内的地址
          - 阿里云 https://mirrors.aliyun.com/goproxy/
          - 七牛云 https://goproxy.cn,direct
        - direct 用于指示 Go 回源到模块版本的源地址去抓取（比如 GitHub 等）
      - GOSUMDB
        - 用来校验拉取的第三方库是否是完整的
        - 默认也是国外的网站，如果设置了GOPROXY，这个就不用设置了
      - GONOPROXY 通过设置GOPRIVATE即可
      - GONOSUMDB 通过设置GOPRIVATE即可
      - GOPRIVATE 通过设置GOPRIVATE即可
        - go env -w GOPRIVATE="git.example.com,github.com/aceld/zinx
        - 表示git.example.com 和 github.com/aceld/zinx 是私有仓库，不会进行GOPROXY下载和校验
        - go evn -w GOPRIVATE="*.example.com"
        - 表示所有模块路径为example.com的子域名，比如 git.example.com 或者 hello.example.com 都不进行GOPROXY下载和校验
    - 通过 go env来查看环境变量
      - go env -w GO111MODULE=on
      - 或者通过 Linux export 环境方式也可以
  - 使用Go Modules初始化项目
    - 1 开启Go Modules模块 保证GO111MODULE=on
      - go env -w GO111MODULE=on
      - export GO111MODULE=on
        - 设置在用户启动脚本中
        - 需要重新打开终端或者执行source ~/.bashrc
    - 2 初始化项目
      - 任意文件夹创建一个项目（不要求在$GOPATH/src） mkdir -p $HOME/aceld/modules_test
      - 创建go.mod文件，同时起当前项目的模块名称 go mod init github.com/aceld/module_test
      - 就会生成一个go mod文件
        - module github.com/aceld/moudles_test
        - go 1.14
      - 在该项目编写源代码
        - 如果源代码中依赖某个库(比如: github.com/aceld/zinx/znet)
        - 手动down go get github.com/aceld/zinx/znet
        - 自动down
      - go mod 文件会添加一行新代码
        - module github.com/aceld/moudles_test
go 1.14
require github.com/aceld/zinx v0.0.0-20200315073925-f09df55dc746 // indirect
        - 含义当前模块依赖github.com/aceld/zinx
        - 依赖的版本是 v0.0.0-20200315073925-f09df55dc746
        - //indirect 表示间接依赖
          - 因为项目直接依赖的是znet包
          - 所以所间接依赖zinx包
      - 会生成一个go.sum文件
        - github.com/aceld/zinx v0.0.0-20200315073925-f09df55dc746 h1:TturbcEfboY81jsKVSQtGkqk8FN8ag0TmKYzaFHflmQ=
github.com/aceld/zinx v0.0.0-20200315073925-f09df55dc746/go.mod h1:bMiERrPdR8FzpBOo86nhWWmeHJ1cCaqVvWKCGcDVJ5M=
github.com/golang/protobuf v1.3.3/go.mod h1:vzj43D7+SQXF/4pzW/hwtAqwc6iTitCiVSaWz5lYuqw=
        - go.sum文件的作用 罗列当前项目直接或间接的依赖所有模块版本，保证今后项目依赖的版本不会被篡改
        - h1:hash
          - 表示整体项目的zip文件打开之后的全部文件的校验和来生成的hash
          - 如果不存在，可能表示依赖的库可能用不上
        - xxx/go.mod h1:hash go.mod文件做的hash
  - 修改项目模块的版本依赖关系
    - go mod edit -replace=zinx@v0.0.0-20200306023939bc416543ae24=zinx@v0.0.0-20200221135252-8a8954e75100
    - go mod文件就会被修改
      - module github.com/aceld/modules_test
go 1.14
require github.com/aceld/zinx v0.0.0-20200306023939-bc416543ae24 // indirect
replace zinx v0.0.0-20200306023939-bc416543ae24 => zinx v0.0.0-20200221135252-8a8954e75100
- 小试牛刀 golang经典的案例 即时通信系统